Operating Systems:

OS is necessary for resource management that means allocation of CPU, GPU, memory and disk.
OS is necessary for not making apps bulky and not violating DRY Principle. That is achieved by abstraction.
OS acts as an interface between the user and the computer hardware.
OS has access to computer hardware.
OS does execute the programs by providing isolation and protection.

Goals of OS:

Maximum CPU utilisation.
No process starvation.
High priority execution.

Types of OS:

Single Process: Processes 1 task at once. Violates point 1, 2, 3. Example: MS DOS
Batch Processing: Similar jobs are batched together. Individual jobs inside batches get processed. Violates all 3.
Multi-programming OS: Single CPU and Queue of jobs. Works on context switching using process control blocks.
Multi-Tasking OS: Single CPU + context switching + time sharing. Max CPU utilisation.
Multi-processing OS: Multiple CPU + context switching + time sharing.
Distributed OS: Loosely coupled multiple machines. Interface for multiple computers connected together. Fault tolerant and provides parallelism.
RTOS: Real time error free execution.

Process:

Program (code) under execution. Program goes from disk to RAM while getting executed.

Thread:

Light weight processes that can be executed independently, but use shared memory to a process.
Each thread has its own TCB(thread control block) for context switching.
Memory in threads is as follows:
stack1, stack2 …. stack3 heap, data, text.

Efficient for multi processing environments so that program can be broken down into independent threads and then get executed in parallel. For single CPU it will take the same time and switch contexts only.

Multi Tasking	Multi Threading
More than 1 process	More than 1 thread
Isolation of memory and protection	No isolation of memory and protection
Processes are scheduled	Threads are being scheduled.
1 CPU	More than 1 preferred.

Process Context Switching	Thread Context Switching
Slow switching	Fast switching
Includes switching of memory and address.	Does not switches memory and address.
CPU cache state is flushed.	CPU cache state is preserved.

Benefits of Multi threading:
1. Responsiveness: Useful in interactive apps, can be easily threaded.
2. Resource Sharing: Threads have shared memory, so no IPC required.
3. Economy: Easy to understand.
4. Multi Core Utilisation: So more scalable.

Components of OS:

User space interacts with kernel and kernel with OS.
1. User space - GUI and CLI
2. Kernel - First part to load

User Space:
Does not have access to the hardware.

Kernel:
Heart of OS, directly interacts with OS.

Shell: (.sh)
Command interpreter which reads commands form the user and gets them executed.
Terminal:
Window for the shell to work in.

How do they interact?
IPC - inter process communication (for communication b/w 2 independent processes.)
2 methods for IPC are Shared Memory and Message Passing.

Functions of Kernel:
1. Process Management - Creation, termination, scheduling, Synchronisation, Communication
2. Memory Management - Allocate / Deallocate, keeping rack of allocated memory.
3. File Management - Create, delete, directory management
4. I/O Management - Input and output devices, pen drives etc (Spooling, Buffering, Caching)

Typesof Kernel:
1. Monolith Kernel: Everything in kernel. switch b/w kernel and user space using software interruption. it is fast.
2. Micro Kernel: I/O and file managed in user space and rest in kernel. more robust.
3. Hybrid Kernel: File manage in user space and rest in kernel. Speed and design of mono and robust from micro.
4. Nano/Exo Kernel: Beyond Scope.

System Calls in Operating System:

A system call is a mechanism using which a user program can request a service from the kernel for which it does not have the permission to perform.
They are implemented in C.

Flow of how it works:
User executes a process. (User space)
Gets system call using the ***"Glibc"*** library. (US)
Executes system call to create a process. (KS)
Return to US.

Types of System Calls:
1. Process Calls: end, abort, load, execute, get and set attributes, wait event, signal event, allocate/free memory.
2. File Management: open, close, delete, create, read, write, search.
3. Device Management: request, release, read, write, attach or detach devices.
4. Information Maintenance: get and set time and date, get and set system data, get file attributes.

5. Communication Management: create and delete connections, send, receive transfer status.

Examples of Windows & Unix System calls:

Category	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Management	CreateFile() ReadFile() WriteFile() CloseHandle() SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	open () read () write () close () chmod() umask(chown()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Management	GetCurrentProcessID() SetTimer() Sleep()	getpid () alarm () sleep ()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe () shmget () mmap()

Different Storages in Computer:

First 3 are primary memory and rest secondary memory ->

1. Register
2. Cache
3. Main Memory
4. Electronic Disk
5. Magnetic Disk
6. Optical Disk
7. Magnetic Tapes

Comparison:

1. Cost: Register is the costliest (Silicon and Transistors).
2. Access Speed: same order.
3. Storage Size: opposite fashion in this.
4. Volatility: Primary flushes but secondary persists after booting.

How OS creates a process:

Programs are in disk.

Steps:

1. Load the program and static data in the memory (RAM).
2. Allocate runtime stack (part of memory used for local variables, function arguments, return values).
3. Allocate runtime heap (part of memory used for dynamic allocation).
4. I/O task (i/p handle, o/p handle, error handle).
5. OS hands off control to main().

Architecture of Processes:
stack heap, data, text

text is compiled code and data means static and global data.
2 famous errors: stack overflow and out of memory: recursion w/o base case and not freeing memory.

Attributes of Processes:
OS lists processes in process table and each of this entry in this table is PCB (Process Control Block).

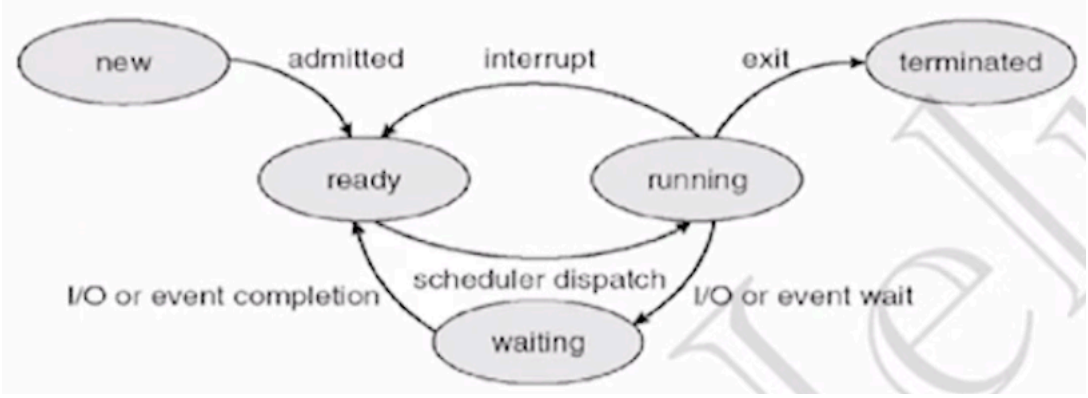
This is a data structure that stores all the data about the process.
PCB:
Process ID: unique identifier.
Program Counter: stores the address of the next machine instruction that the process should execute.
Process State: Current state (new, waiting, running).
Priority: Simple priority.
Register: Stores the CPU registers so that we can easily know the last state of process.
Open File List: Opened files at current execution.
Open Devices List: IO devices that were open at last execution.

Different Process States:
1. New: program becomes process.
2. Ready State: is sitting in memory in ready queue.
3. Running: CPU allotted.
4. Waiting: Waiting for IO completion.
5. Termination: Process finished.

New to Ready: Job queue when in new and transferred via job scheduler / long term scheduler. (LTS)
Ready to Running: Ready queue when in ready and transferred via Short term scheduler / CPU scheduler. (STS)
These are called short and long based upon frequency of task. like STS has to do tasks very quickly in ms.

The short-term scheduler selects a process from the ready queue to execute next, whereas the dispatcher gives the CPU to the selected process by performing the context switch, switching modes if required, and transferring control to the process's next instruction.

Degree of multi programming is the no of jobs in ready queue, which is affected / managed by LTS.
LTS should choose good mix of processes that means: not all IO or not all CPU bound processes.
If ready queue has memory intensive tasks and so some processes from ready queue get swapped out to secondary memory and later loaded back, this is done using Medium Term Scheduler.



Context Switching in OS:

Done by Kernel and using PCB's for contexts of processes.
Done before. Learn about counter and CPU registers a bit.

Orphan Process:
Using fork() command we make child processes of a parent.
the ultimate parent (1st process of OS) process is ***init*** which has a PID of 1.

When P1 is running and creates P2 using fork() and P1 gets exception and terminates, then OS assigns P1 which is currently an orphan process, init as P1's parent.

Zombie Process:
Process whose execution is completed but still has entry in the process table. (ps and see hidden using ps -a).
Read more on this.

Process Scheduling:

The process of providing CPU to process based on some logics/algorithms.

Types of processes:
1. Preemptive: Leaves CPU when termination, IO or when time quantum expires.
2. Non-preemptive: Leaves CPU only on termination or IO task. (time sharing not possible here)

Starvation: Process is waiting for CPU, more when there are more non-preemptive processes.
Overhead: more when more preemptive processes due to requirement of context switching.
CPU Utilisation: more when more preemptive tasks are present, as no ideal time.

Goals of CPU Scheduling:
1. Max CPU utilisation: Simple to understand.
2. Least Turn Around Time (TAT): Time from first time in ready queue to end. (CT - AT).
3. Least Wait Time (WT): Time process spends waiting for CPU (TAT - BT).
4. Least Response Time (RT): Time to get CPU for the first time after being in ready queue.
5. Max Throughput: Processes completed per unit time.

Other terms:
Arrival Time: Time of arrival of process in the ready queue for the first time.
Burst Time: Total time needed by the process for execution.
Completion Time: Time when process gets executed completely.
Convoy Effect: When larger BT processes comes first so smaller BT processes have to wait for long. Average WT increases. occurs in FCFS, non preemptive SJF, both priority scheduling.

SCHEDULING ALGORITHMS:

1. First come First serve (FCFS): Early arrival time gets executed first. Causes ***convoy effect.***
2. Shortest Job First (SJF): Least AT then Least BT will get CPU. non preemptive and preemptive both versions are there. practically not possible as we don't really know BT of processes. preemptive SJF can not have convoy effect as process arrives and we stop the on going higher BT and carry on with lower BT.
3. Priority Scheduling: Highest priority first, both versions here also. Issue of convoy (indefinite wait time) solved using aging, increasing priorities gradually over time
4. Round Robin: AT + TQ is the criteria, idea is preemption + FCFS, designed for time sharing systems, overheads of context switching are high.
5. Multi Level Queue Scheduling: Ready queue is divided into multiple queues and based on a process is a System process, Interactive (foreground) process or background process it is assigned a queue and once assigned it can not be assigned to another queue and each queue has its own scheduling algorithm. And preemptive priority scheduling is used to schedule

among different sub-queues, with highest priority of system process queue and lowest for background process queue. Starvation present.

6. Multi Level Feedback Queue: Multiple sub queues but processes can change among different queues. higher burst time processes get assigned to lower level of queues and IO and interactive processes move in high level of queue. Aging method is used to prevent starvation. It is a flexible model and is designed in following steps:

- no of queues
- scheduling algorithm of each queue
- how to promote a process
- how to demote a process
- how to assign first queue

	FCFS	SJF	PSJF	Priority	P-Priority	RR	MLQ	MLFQ
Design	Simple	Complex	Complex	Complex	Complex	Simple	Complex	Complex
Preemption	No	No	Yes	No	Yes	Yes	Yes	Yes
Convoy effect	Yes	Yes	No	Yes	Yes	No	Yes	Yes
Overhead	No	No	Yes	No	Yes	Yes	Yes	Yes

Concurrency:

Concurrency is doing multiple tasks at overlapping time intervals not exact same time. example: CPU context switching, whereas Parallelism is doing multiple tasks at exact same time. example: multiple CPUs.

Race Conditions in OS:
A problem may occur when multiple processes or threads concurrently or in parallel access and modify some shared data, and final result differs on their execution order.
This part of program from where this shared data is accessed and modified is known as ***critical section.***

Solutions:

1. *Atomic Operation:* Do the whole critical prat and then move forward.
2. *Mutual Exclusion using Locks*: No 2 threads can access at same time.
3. *Conditional Variables*: Used when working of 1 thread depends upon task to be done by thread 2. Avoids busy waiting, by acknowledging thread 1 that something has changed so continue executing.
4. *Semaphores*:

Conditions for Solutions to work:

1. Mutual Exclusion: No 2 threads can access at same time.
2. Progress: Equal opportunity for each thread when there is no one in critical section.
3. Bounded waiting: indefinite waiting should not occur.

Can simple flag work? NO
it does bring mutual exclusion but no progress.
so for that there is Peterson's solution. but limited to 2 threads.

Problems of locks:

1. *Contention:* indefinite waiting for other threads that were unable to enter.
2. *Deadlocks:* resource already allocated someone else.
3. *Debugging Issue*: Difficult.
4. *Starvation:* Late arriving high priority job have to wait.

Producer Consumer Problem: (Bounded Buffer Problem)

1 producer thread and 1 consumer thread

Reader-Writer Problem:
Some read threads and some write threads.

if more than 1 reader -> no issue.
if more than 1 writers / 1 writer and other thread are any read or write -> Semaphore.
1. Mutex (Binary Semaphore): To ensure mutual exclusion when read count is updated, no 2 threads modify this at same time.
2. Write (Binary Semaphore): Common for both reader and writer.
3. Read Count (integer): Maintains count of readers.

Dinning Philosopher Problem:

Deadlocks:

Necessary conditions for deadlock:
1. Mutual Exclusion: P2 has to wait for P1 to release R1 to acquire R1.
2. Hold and Wait: until work is not done resource must be locked by P1.
3. No preemption: not released till execution of allocated process is completed.
4. Circular Wait: allocation and demand is in circular fashion.

Resource Allocation Graph:
cycle -> may be dead lock.

Handle deadlock:
1. Protect or avoid
2. Allow to go in deadlock, detect, solve
3. Ostrich Algorithm

Prevention Techniques: (do not let to go in deadlock)
Make any of the 4 conditions false.
1. Sharable resources like read only files can be used for violating mutual exclusion.
2. Allocate ALL resources before execution starts nd allow process to req only when it has 0 resources.
3. release resources and wait for them to be free and lock all at same time when available.

Avoidance techniques: (deadlock occurred, now what?)
1. *Bankers Algorithm*: We need to schedule processes in such a way that they have to end up in safe states.

Deadlock Detection: (we let go system in deadlock and then recover it afterwards)
1. Single instance of each resource (wait for graph)
2. Multiple instance: Bankers algorithm

Recovery from deadlock:
1. Process Termination: abort process of low priority.
2. Resource Preemption: free resource causing deadlock.

Memory Management in OS:

OS allocates a *base* in RAM (physical memory space) and stores *offset* for each process.

Physical Address	Logical Address
address loaded in the memory register of physical memory	address used by any process

Generated by MMU	Generated by CPU
user can't access this	user can access this
R + 0 to R + max (R is base)	Virtual Address (0 to max)

Types of allocation:
1. *Contiguous:* Complete process gets memory at one time. Fixed Partitioning (blocks divided have equal size - suppose there are 4MB partitions and 5 processes come 3MB each, i will not be able to give 4MB to last process although its available - this is called *external fragmentation*, And the wastage to 1MB in each allocation is termed as *internal fragmentation* Also if a process of 5MB comes then this can't be loaded. Also degree of multiprogramming is very less in this scenario), Dynamic Programming (Variable size blocks decided at time when process comes, this still has external fragmentation issue.)
2. *Non - contiguous*: Linked List is used to store the free space available. Solves external fragmentation.

CPU does *de-fragmentation* (abstract function) to minimise external fragmentation. Base address changes and offset remains same.

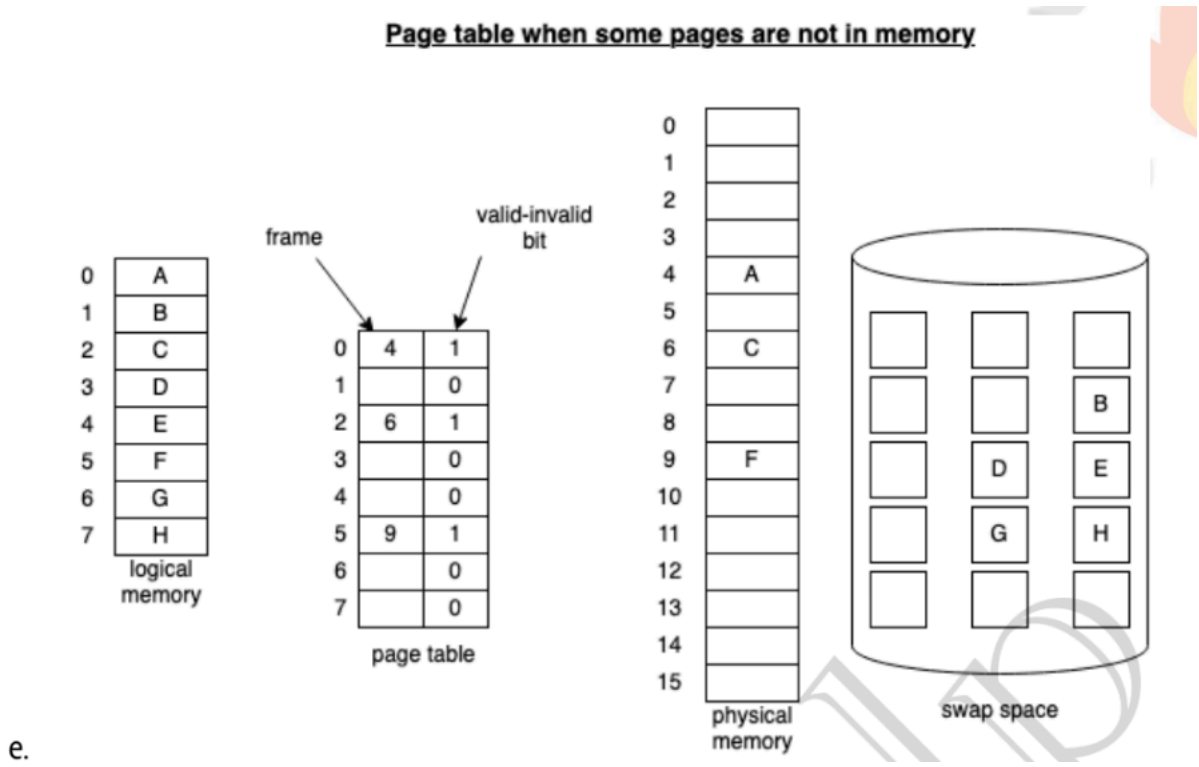
How to assign process the blocks of free space?
1. *First Fit*: First capable node of free list.
2. *Best Fit*: Smallest capable segment to fit the process.
3. *Next Fit*: Iterate from where ever the counter was left.
4. *Worst Fit*: Largest hole to accommodate.

Paging:
A method to do non-contiguous allocation taken in action by MMU.
Page is basically the fixed size division of a logical space address.
This fixed size partition is also done in RAM to fit page's fixed division in it, which is called frame.
Each process has its own page table and is loaded in RAM only.

CPU manages it in PTBR register. (Page table base register). Used during context switching.
There are a lot of memory references in this so paging is slow, so we have to do *caching.*
Translation look-ahead buffer: A cache table used to find the frame number corresponding to the page number. It is common for multiple processes as it stores PID, Page number and Frame number, So we do not need to flush it when process switches.

Segmentation:
Variable partitioning of logical address space. So here MMU maintains a segment table rather than a PT.
Logical address divided into segment number and offset.

Virtual Memory: a technique to execute processes that are not in physical memory. A part of secondary memory is treated as main memory called sap space. Only the needed pages of a program are loaded in the main memory.

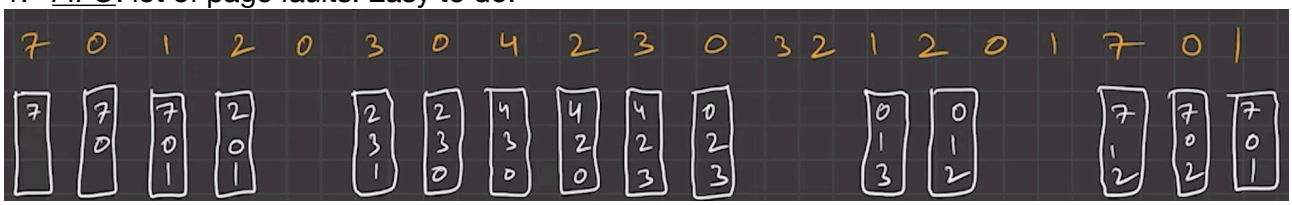


e.

Page Replacement Algorithm:
Page faults occur when a page is demanded and it is not currently in the physical memory and needs to be loaded from the virtual memory.

Who brings page in? Pager.
Who brings complete processes in? Swapper.
How to decide which page should be swapped? Page Replacement Algorithm.

Page Replacement Algorithms:
1. *FIFO*: lot of page faults. Easy to do.



2. *Optimal Page Replacement*: Impossible to implement. We remove page from frame which will be needed late.
3. *Least Recently Used*: simple.
4. *Least Frequently Used*: simple.
5. *Most Frequently used*: simple.

Thrashing:
When CPU is more busy serving page faults rather than executing processes. occurs usually in high degree of multiprogramming.

Precautions:
1. *Working set model*: Allocate all locality pages of a process to RAM at once, but if number of frames are less than size of current locality process is bound to thrash.
2. *Page Fault Frequency*: Set a lower and upper bound on number of pages from each process to occupy the frames.

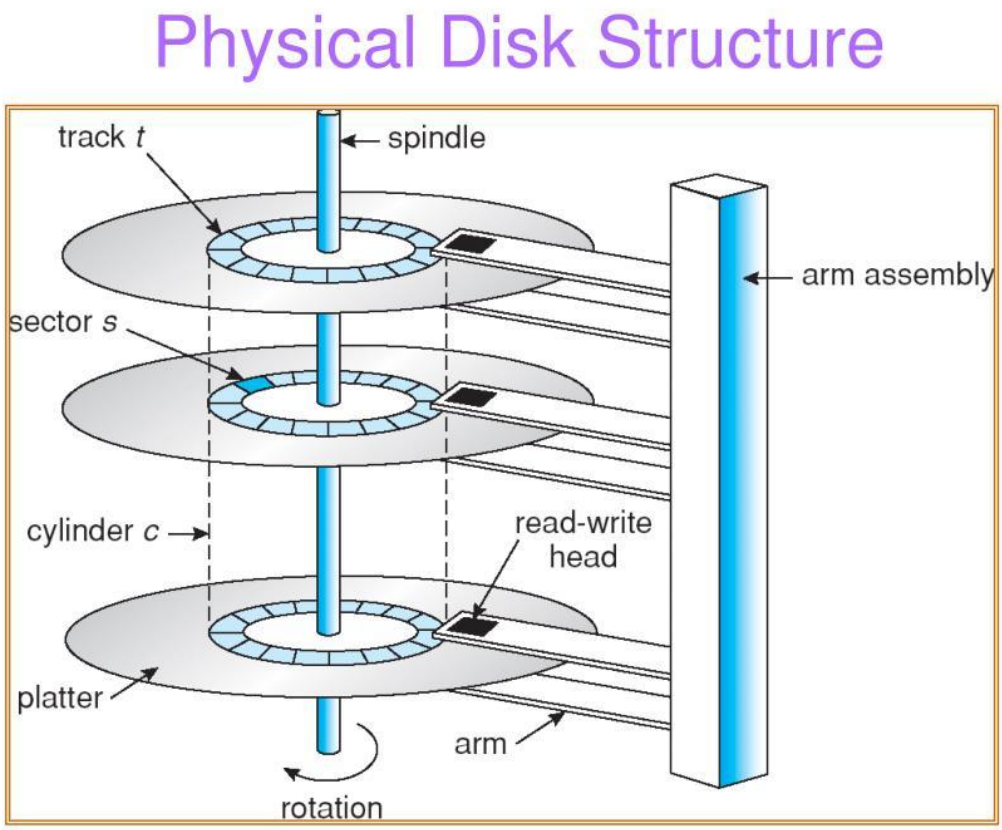
Disk Scheduling:
1. What a disk physically/logically is

2. How an OS accesses data on disk
3. How the OS decides which disk request to serve next — disk scheduling

A **disk** is a secondary storage device used to store data **persistently**. eg - HDD (Hard Disk Drive), SSD (Solid State Drive)

RAM	Disk
Volatile	Non-volatile
Loses data on power off	Retains data
Very fast	Much slower
Expensive per GB	Cheaper per GB
Directly used for active programs	Used for persistent storage

We generally talk about HDD as they are accessed by mechanical movement of a disk head.



Platter - Circular magnetic disks where data is stored, each has 2 surfaces (top n bottom), there may be multiple of these in a hard disk.
Spindle - This rotates all platters, higher RPM generally means lower rotational latency.
Read/Write head - each platter surface has one of these, moves across surface using *actuator arm*.
Actuator Arm - moves head between tracks, major delay - *seek time*.
Track - A circular path on a platter surface. (many concentric on 1 surface)
Sectors - A track is divided into sectors. (have physical storage blocks)

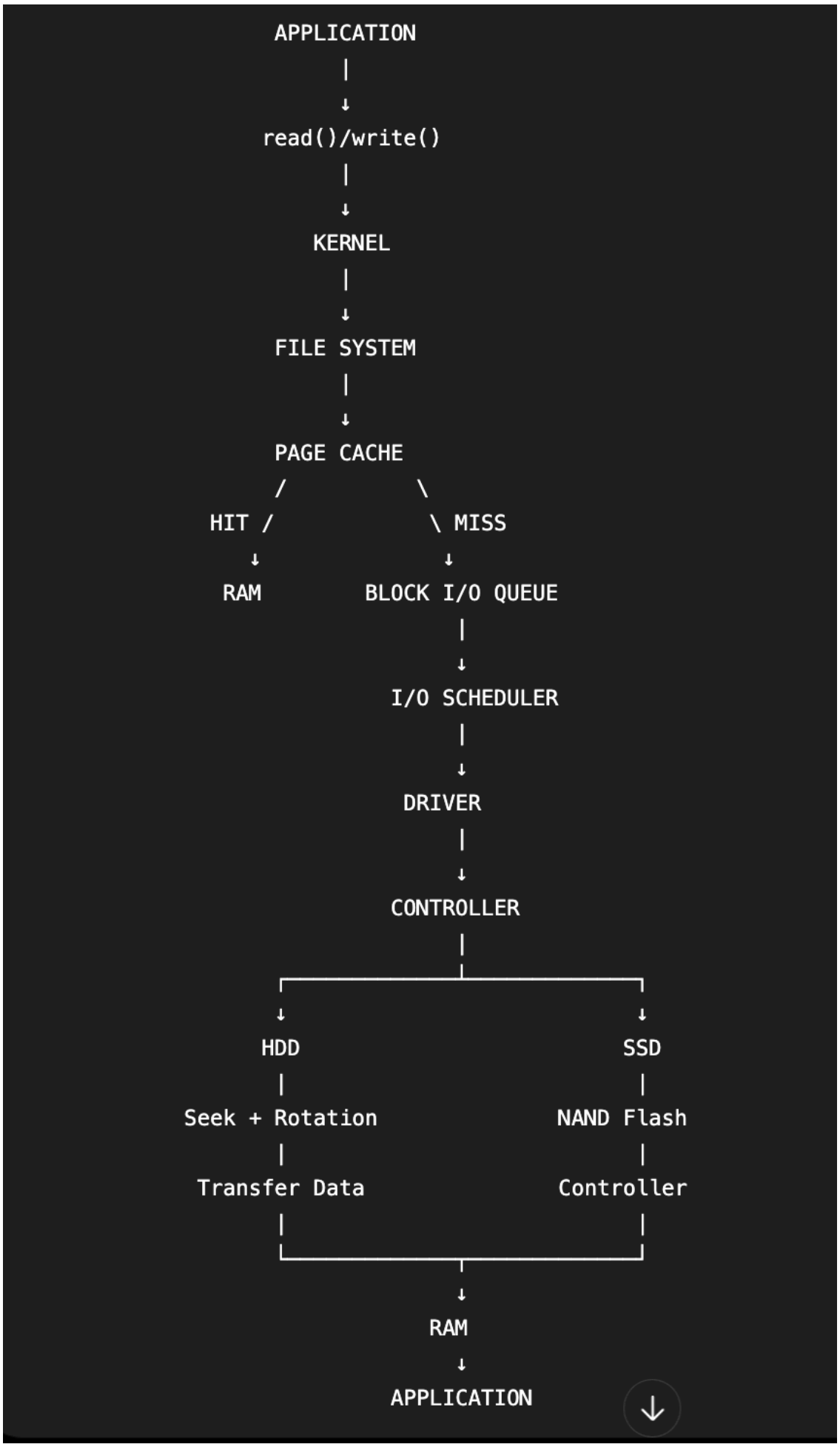
Disk Access Time = *Seek Time* + *Rotational Latency* + *Transfer Time* + *Controller/queue overhead*

Time	Meaning
Seek Time	Move head to target track
Rotational Latency	Wait for sector to rotate under head
Transfer Time	Actually transfer data
Disk Access Time	Overall time to access data
Response Time	Request submission → response
Queueing Time	Time waiting in request queue

Now there are a few algorithms of disk scheduling by which we judge which request should be served next:

1. *FCFS* - first come first serve, simple, fair, no starvation, bad average seek time, lot of movements.
2. *SSTF* - shortest seek time first, choose req closest to head, can cause starvation for far away track.
3. *SCAN* - Elevator algo, go towards predefined direction, goes till extreme end if requests are remaining in queue, starvation and bad average in case of dynamic adding of requests.
4. *C-SCAN* - circular scan, scans in one direction only and starts from 0 again, more uniform waiting time.
5. *LOOK* - moves in both direction only till last request.
6. *C-LOOK* - this moves only in one direction and only till last request.

Algorithm	Main Idea	Starvation?	Head Movement
FCFS	Arrival order	No	High
SSTF	Closest request	**Yes**	Low
SCAN	Elevator	No	Moderate
C-SCAN	Circular elevator	No	Moderate
LOOK	SCAN without unnecessary end travel	No	Lower
C-LOOK	C-SCAN without unnecessary end travel	No	Lower


APPLICATION
read()/write()
KERNEL
FILE SYSTEM
PAGE CACHE
HIT
MISS
RAM
BLOCK I/O QUEUE
I/O SCHEDULER
DRIVER
CONTROLLER
HDD
SSD
Seek + Rotation
NAND Flash
Transfer Data
Controller
RAM
APPLICATION

Other Terms:

Virtual memory
Cache vs Buffer
Page Cache
Page Faults
How does data access take place?
DMA - Direct memory Access

56. What Happens During a File Read?

Interviewers may ask:

> "What happens when I read a file?"

A strong answer:

```
Application
   ↓
read() system call
   ↓
Kernel
   ↓
File system checks file metadata
   ↓
Check page cache
   ↓
If cache hit → return data
   ↓
If cache miss
   ↓
Generate block I/O request
   ↓
Block layer queues request
   ↓
Device driver/controller
   ↓
Storage device
   ↓
Data transferred to memory
   ↓
I/O completion
   ↓
Process receives data
```